IntentChecker: Validating Developer Intentions in Solidity via Intent Annotations to Mitigate Numeric Logic error

Inseong Jeon[1], Sundeuk Kim[1], Hyunwoo Kim[1], Hoh Peter In[1*]

[1*]Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Logic errors in smart contracts—where program behavior diverges from developer intent—are among the most challenging vulnerabilities to detect. Unlike well-defined patterns such as reentrancy or integer overflow, logic errors require knowledge of developer intent, which is absent from the code itself. Existing approaches, including static analyzers, dynamic testing, and LLM-based methods, cannot reliably identify such errors without explicit intent representation. This paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during smart contract development. IntentChecker introduces an intent annotation model with two annotation types: `@During` annotations specify intended relationships at specific program points within a function, while `@Post` annotations specify intended relationships after function execution. The underlying analysis uses interval-domain abstract interpretation to compute value ranges at each program point and check whether they satisfy the specified intentions. We evaluated IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions and providing immediate feedback during development.

Keywords: Smart Contract Development, Solidity, Numeric Logic Error, Intent Annotations, Abstract Interpretation, Developer Tools

1 Introduction

Smart contracts are immutable once deployed, making security assurance during development critical. Among vulnerability types, logic errors present a distinct challenge: they arise when program behavior diverges from developer intent, a discrepancy that is inherently invisible to compilers and external reviewers alike (Soud et al., 2024).

A recent empirical study (Chaliasos et al., 2024) confirms this challenge in practice. Practitioners identified logic errors as the most difficult vulnerability type to detect, reporting that existing security tools are least effective against them. This gap exists because current approaches—static analyzers, dynamic testing, and even recent LLM-based methods—predominantly target well-defined vulnerability patterns. Logic errors, however, require knowledge of developer intent, which is absent from the code itself. Various research efforts have attempted to detect specific logic errors, but without explicit intent representation, they must rely on predefined detection patterns. Such patterns can identify violations of specific rules, but not actual deviations from intended behavior. Consequently, even when these tools report no issues, developers cannot be certain that their code behaves as intended.

(Chaliasos et al., 2024) also provides insight into developer needs. Developers expressed a preference for development-integrated tools such as IDE extensions and linters over standalone analyzers. Moreover, when asked about the most valued properties of security tools, developers prioritized low false negatives (92.3%) over low false positives (50%), indicating a strong preference for tools that do not miss real issues, even at the cost of occasional false alarms. This suggests a need for development-phase tools that can provide sound guarantees about code behavior. To determine where such intent specification is most needed, we analyzed 1,023 non-trivial functions from DAppSCAN-source (Zheng et al., 2024) (see Table **??**). Numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic —handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations becomes essential.

Based on these observations, this paper presents IntentChecker, a tool that enables developers to specify and validate their intentions for numeric operations during the development phase. IntentChecker introduces an intent annotation model consisting of two annotation types: `@During` annotations specify intended relationships that should hold at specific program points within a function, while `@Post` annotations specify intended relationships that should hold after the function completes execution. Developers write these annotations alongside their code, and IntentChecker validates whether the program's abstract execution satisfies these specified intentions.

The underlying value analysis is performed using interval-domain abstract interpretation, building upon our previous work on SolQDebug (Jeon et al., 2025), an interactive debugger for Solidity. Through `@Debugging` annotations, developers specify value ranges for variables, and the analyzer computes abstract values at each program point. IntentChecker extends this capability by checking whether these computed values satisfy the developer-specified intent annotations, providing immediate feedback during development rather than requiring completed code.

This paper makes the following contributions:

- We present an intent annotation model that enables developers to express intended numeric relationships at both statement-level (`@During`) and function-level (`@Post`) granularity.
- We develop validation algorithms for each annotation rule that leverage interval-domain abstract interpretation to check whether program behavior satisfies specified intentions.
- We evaluate IntentChecker on real-world smart contracts, demonstrating its effectiveness in validating developer intentions during the development phase.

The remainder of this paper is organized as follows. Section 2 provides background on numeric logic errors and their classification by execution semantics. Section 3 describes the design of IntentChecker, including its intent model and validation engine. Section 4 presents our evaluation results. Section 5 discusses limitations and future work. Section 6 reviews related work, and Section 7 concludes.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research efforts have introduced various specifically-named logic errors in smart contracts (Chen et al., 2025; Sun et al., 2024; Soud et al., 2024; Zhang et al., 2023; Zhou et al., 2023). However, existing approaches to logic error classification predominantly follow a vulnerability-pattern-based taxonomy, categorizing issues by their manifestation such as division-by-zero, precision-loss, integer overflow, and similar named patterns. While this approach enables systematic enumeration of known vulnerability types, it suffers from a fundamental limitation: it focuses on vulnerability phenomena rather than directly representing code behavior. Such classification describes what went wrong in terms of observable symptoms, but provides insufficient connection to the actual execution dynamics that produced those symptoms. Consequently, while these taxonomies can identify "what is incorrect," they offer limited insight into "how the code actually behaves" during execution.

This limitation becomes particularly problematic when attempting to validate whether code behavior aligns with developer intentions. Vulnerability-pattern-based approaches can detect known manifestations of logic errors, but cannot systematically address the broader question of whether the implemented logic correctly reflects the developer's intended computation. For instance, a "precision-loss" classification indicates that numerical precision was reduced during computation, but provides no framework for determining whether this reduction was intentionally designed, accidentally introduced, or falls within acceptable bounds according to the developer's original intent.

To address these limitations, we propose an execution semantics-based approach to logic error classification. Rather than categorizing errors by their symptomatic manifestations, this approach systematically represents code behavior according to function execution dynamics. By focusing on how code actually operates during execution—encompassing input processing, computation steps, side effects, and output

generation—this classification enables direct comparison between implemented behavior and developer intentions. This execution-centric perspective provides a structured framework for systematically validating whether code behavior matches intended logic across all critical points in function execution.

We define numeric logic errors within this execution semantics framework as follows: *bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer's intended numerical or state relationships during function execution.*

It is crucial to distinguish between single-transaction and multiple-transaction contexts when analyzing logic errors. Single-transaction errors manifest within the execution of a single function call—where input validation, computation, side effects, and output generation occur in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples include Risky First Deposit and Price Manipulation by AMM vulnerabilities (Sun et al., 2024; Zhang et al., 2023), where attackers exploit state changes across multiple function invocations to manipulate contract behavior. Our approach focuses on single-transaction function execution, as this aligns with the development-phase validation model where developers can immediately verify individual function behaviors against their intentions. This scope enables precise, actionable feedback during implementation rather than requiring complex multi-transaction scenario analysis.

Figure 1 illustrates our function execution semantics classification framework. Each function execution progresses through four distinct stages: (1) *INPUT* represents parameters passed to functions—both at the contract function level and internal function calls; (2) *COMPUTATION* executes statements and evaluates expressions that transform input data through arithmetic, logical operations, and control flow; (3) *SIDE EFFECTS* captures interactions beyond local computation, including internal and external function calls that may modify contract or external state; and (4) *OUTPUT* represents the function's intended effects—including return values, state variable updates, and notably, missing or incorrect outputs where computed values fail to properly update state or return. This classification enables systematic mapping of error patterns to their manifestation points in the execution flow, providing a more precise understanding of where and how numeric logic errors occur within single-transaction contexts.

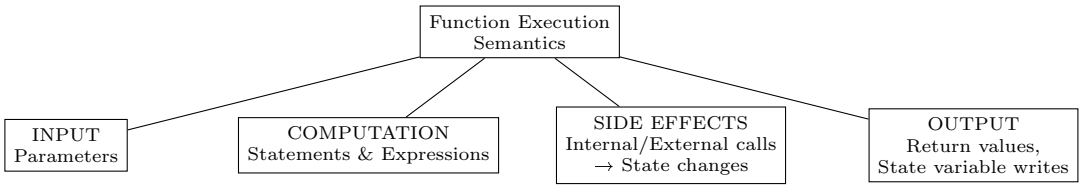

Fig. 1: Function Execution Semantics Classification

Table 2: Mapping of Existing Error Patterns to Execution Semantics

Execution Stage	Error Pattern	Reference
INPUT	Minor Amount Retention	(Chen et al., 2025)
COMPUTATION	Div In Path	(Chen et al., 2025)
	Operator Order Issue	(Chen et al., 2025)
	Exchange Problem	(Chen et al., 2025)
	Precision Loss Trend	(Chen et al., 2025)
	Wrong Interest Rate Order	(Sun et al., 2024)
	Wrong Checkpoint Order	(Sun et al., 2024)
	Erroneous Accounting	(Zhang et al., 2023)
	Arithmetic Mistakes[a]	(Zhou et al., 2023)
SIDE EFFECTS	Arithmetic Mistakes[b]	(Zhou et al., 2023)
OUTPUT	Greedy Contract	(Soud et al., 2024)

[a] e.g., wrong reserve ratio calculation (MerlinLab), precision loss (ValueDeFi)
[b] e.g., stale pool state used after update (CoverProtocol)

Table 2 demonstrates how existing named error patterns from recent research map to our execution semantics framework. For instance, Minor Amount Retention occurs in the INPUT stage when function parameters fail validation checks (e.g., divisibility constraints). COMPUTATION errors like Wrong Interest Rate Order manifest when calculation sequences produce incorrect results due to improper ordering. SIDE EFFECTS errors such as stale state usage arise when internal function calls create inconsistent state updates. Finally, OUTPUT errors like Greedy Contract occur when computed values fail to properly update state variables as intended. This mapping reveals that most identified patterns concentrate in the COMPUTATION stage, highlighting where verification efforts should focus.

References

Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3623302

Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). https://doi.org/10.1109/TSE.2025.3555622

Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). https://doi.org/10.21203/rs.3.rs-8077153/v1

Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. Empirical Software Engineering 29(1), 13 (2024). https://doi.org/10.1007/s10664-023-10398-5

Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). https://doi.org/10.1145/3597503.3639117

Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). https://doi.org/10.1109/ICSE48619.2023.00061

Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. IEEE Transactions on Software Engineering 50(6), 1360–1373 (2024). https://doi.org/10.1109/TSE.2024.3383422

Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023). https://doi.org/10.1109/SP46215.2023.10179435